

GISMA

University of Applied Sciences

M604 Advanced Programming

Simple Task Manager API Project Report

Student Information

Full Name	Manoj Kumar
Student ID	GH1037914
Module	Advanced Programming M604
Submission Date	18.09.2026

Project URL's

GitHub Repository	https://github.com/manojkumar-techdev/Task-Manager-Java-Project
Video Demonstration	https://drive.google.com/file/d/14Wh_2c8v9Yh-qGEi0HuxP2kCrUMKQvzd/view?usp=sharing

1. Introduction

This project aim to build and involved designing and developing a backend application to handle the data with these things operations, including CRUD (Create, Read, Update, Delete), and also it provide the RESTful API and also the simple GUI interface to the data and the user, as well as strong error handling and validation.

The Task Manager application meets these needs. It is the Java 17 and also the SpringBoot 3.2.5 based web application is used in it. They are able to create the view, update, and also delete the tasks via the RESTful API and also the browser based UI. This project also showcases how do we utilize OOP principles, and how to integrate it with the databases using and also with JPA/Hibernate, plus how to control global exceptions.

Objectives:

Create a Java backend using Spring Boot.

Implement an relational database with JPA.

Use a RESTful API to implement all the CRUD operations.

Make a GUI that runs in a browser.

Use holistic error-handling.

Make use of OOP concepts throughout.

2. Literature Review

Java language is the object oriented oop language, and also the class based, this language that using the concepts of encapsulation, inheritance and also the polymorphism and abstraction. The 'no-code' philosophy of Spring Boot in enterprise app dev - embedded servers, starter deps (Walls, 2016). Spring Data JPA is based on Java Persistence API and offers repository abstractions without the proliferation of data access code.Spring Data JPA is based on the Java Persistence API, and eliminates boilerplate data access code. The standard HTTP verbs act upon resources identified by URIs, and this architecture is known as REST, a creation of Roy Fielding. Spring offers **@ControllerAdvice** for handling exceptions on the global level.

3. System Architecture

The application has a layered design:

Layer	Component	Responsibility
Presentation	TaskController	Handles HTTP requests
Business Logic	TaskService	Apply rules to Input
Data Access	TaskRepository	Interacts with database
Model	Task	Data entity
Error Handling	GLobalExceptionHandler	It Catches all the execeptions
Configuration	DataInitializer	Seeds Sample Data

This is very important and also very useful to have the application that registers the data with the Configurator of that generates the same data for the sample Schema Setup and the CQL, or also the Seeds file.

Requests Flow In This Way: Client to the Controller and then to the Service and then to the Repository of the Database. The Exceptions is bubble up to the global handler which they returns in the simple structured JSON.

Using that Constructor based Dependency the Injection to the promote loose coupling and also testability.

4. Data model and schema

The basic entity is Task, linked to the tasks table:

Code In Java:

```
@Entity
@Table(name = "tasks")
public class Task {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @NotBlank @Size(max = 120)
    @Column(nullable = false, length = 120)
    private String title;
    @Size(max = 500)
    @Column(length = 500)
    private String description = "";
    @Pattern(regexp = "pending|in_progress|done")
    @Column(nullable = false, length = 20)
    private String status = "Pending"?
    @Pattern(regexp = "low|medium|high")
    @Column(nullable = false, length = 20)
    private String priority = "medium";
    @Column(name = "created_at", nullable = false)
    private LocalDateTime createdAt = LocalDateTime.now();}
```

Attribute	Type	Constraints
Id	BIGINT	PRIMARY,KEY AUTO INCREMENT
title	VARCHAR(120)	NOT NULL
description	VARCHAR(500)	NULLABLE
status	VARCHAR(20)	NOT NULL
priority	VARCHAR(20)	NOT NULL
created at	TIMESTAMP	NOT NULL

Also with that the hibernate auto generates and also with the schema also with the **spring.jpa.hibernate.ddl-auto=update**. And the project aim also includes is that the support of to H2 (in the memory, default) and SQLite (persistent, optional) can be switched with **application.properties** only also.

5. RESTful API Design

URL	Verb	Description	Status codes
/api	GET	The API info	200
/api/tasks	GET	List of all the (optional ?status=)	200
/api/tasks/{id}	GET	Get one task Done	200 and 404
/api/tasks	POST	Create the task Done	201 and 400
/api/tasks/{id}	PUT	Update the task Done	200,400 and 404
/api/tasks/{id}	(DELETE)	Delete the task Done	200 and 404

Example to Create:

bash

POST /api/tasks

```
{"title":"Study Java","status":"pending","priority":"high"}
```

In Reply to the question: **201 – Created**: Returned the entire task object formatted as a JSON.

Next came REST conventions – resource-based URIs, proper HTTP verbs, reasonable status codes, and JSON payloads, as well as statelessness.

6. User Interface

GUI is implemented in a single HTML file (**index.html**) which calls the API using only basic JavaScript and fetch. It is set as a shared resource at /. in Spring Boot.

Features:

1. Create form for adding tasks that includes: Title, Description, Status, Priority.
2. In addition to a To-Do list, record metadata with it.
3. Mark Done button.

4. Removes an item with confirmation.
5. Success/error status messages.

7. Error Handling

The 2 examples below are thought to be custom exceptions which extend **RuntimeException** (showing inheritance):

Exception	Trigger	Status
TaskNotFoundException	Invalid Task ID	404
InvalidInputException	Bad status/priority	400

In the **GlobalExceptionHandler**, exceptions are captured using **@ControllerAdvice** and **@ExceptionHandler** for 5 types of exceptions: not-found, invalid-input, bean-validation, malformed JSON, and generic—and return the same object:

json

```
{
  "status": 404,
  "error": "Task not found with id: 999",
  "timestamp": "2026-09-15T23:20:00"
}
```

There is the three levels of the validation for the client side (HTML validation) to do, service-side (manual validation) and bean validation (that also using the **@NotBlank**, **@Pattern** annotation).

8. Technologies

Technology	Version	Purpose
Java	17 (LTS)	Programming language
Spring Boot	3.2.5	Backend framework
Spring Data JPA	3.2.5	database abstraction feature.
Hibernate	6.4.4	ORM implementation
H2 / SQLite	2.2.224 / 3.45.1	Databases
Maven	3.9.16	Build tool
Tomcat	10.1.20	Embedded server

9. Development Methodology

Iterative was adopted in this process, setup, scaffolds, ddl, business logic, api, error handling, ui, test. Project structure:

task-manager-java/

```
|— pom.xml
└— src/main/
    |— java/com/m604/taskmanager/
    |   |— TaskManagerApplication.java
    |   |— config/DataInitializer.java
    |   |— controller/TaskController.java
    |   |— dto/{ErrorResponse, TaskRequest}.java
    |   |— exception/{GlobalExceptionHandler,
    |       |       InvalidInputException,
    |       |       TaskNotFoundException}.java
    |   |— model/Task.java
    |   |— repository/TaskRepository.java
    |   └— service/TaskService.java
    └— resources/
        |— application.properties
        └— static/index.html
```

Run:

bash

cd C:\Users\DELL\Desktop\task-manager-java

mvn spring-boot:run

After running above 2 commands then the server has been started on the <http://localhost:5000> link which is the localhost open it in any browser which you want to run the application with the GUI.

10. Testing and Results

#	Test	Expected	Status
1	Create task (valid)	201 Created	✓
2	Create without title	400 Bad Request	✓
3	Create with bad priority(None)	400 Bad Request	✓
4	Get all tasks	200 OK	✓
5	Get task by valid ID	200 OK	✓
6	Invalid ID was provided for a task!	404 Not Found	✓
7	Update task	200 OK	✓
8	Delete task	200 OK	✓
9	Filter by status	200 OK	✓

A malformed JSON response was received.Bad Request – The JSON response was malformed.

Sample — 404 test:

bash

```
curl -i http://localhost:5000/api/tasks/999
```

After running that one command which written above then the server is not giving the back any response which is basically code error of 404. And All the tests are passed.

12. Conclusion and Future Work

The Task Manager project is a successful implementation of all the necessary components of M604:

Backend Development: using controllers and services in Spring Boot.

Data Interaction: Collection review of reading data with auto schema (JPA/Hibernate)

API Development: Capable of developing an API based environment that is RESTful and an HTML GUI.

Error Handling: Global handler with custom exceptions

Applied of the following core topics of the Java programming language in the OOP section the principles are: The encapsulations, inheritance, interfaces and the exception handling and also done the dependency injection of OOP core topics in Java programming language.

Enhancements of this project is will be done in the future which are these: In future Docker containerization and Spring Security and also the Pagination, Unit tests with the JUnit, Swagger documentation and also the cloud deployment will be done in the future to improve or expand this project in the future.

References

1. Schildt, H. (2014). The complete reference, 9th edition, Java. McGraw Hill Education.
2. Walls, C. (2016). Spring Boot in action 1st ed. Manning Publications.
3. Walls, C. (2015). Spring in Action, 4th Edition. Manning Publications.
4. Walls, C. (2022). Spring in action (6th ed.). Manning Publications.
5. Webb, P., Syer, D., Long, J., Nicoll, S., Winch, R., Wilkinson, A., Overdijk, M., Dupuis, C., Deleuze, S., Simons, M., Pavić, V., Bryant, J., Bhave, M., Meléndez, E., Frederick, S., & Halbritter, M. (2024).
6. Spring Data JPA. (2026). JpaRepository [API documentation].
7. These above documents has been downloaded from the online websites list.This documentation is for the Spring Boot reference documentation.RC1). (2015).
8. Project checkpoint #1 [Course assignment]. (2022). Main/Main: Final Project & Exam.